06 - Backtracking

Algorithms & Complexity

Eduard Torres Montiel - eduard.torres@udl.cat

Logic & Optimization Group, University of Lleida

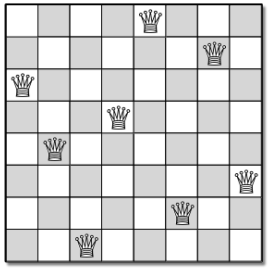
23/04/2025

Practical case: the N-queens problem

Consider the n-queens problem.

In this problem we have an $n \times n$ chess board where we must fit n queens.

The constraint is that **none of the queens can attack each other** (i.e. no two queens are in the same row, the same column, or the same diagonal).



Example solution for the 8-queens problem.

From the problem statement we derive that there must be **exactly one queen** at each row, so our approach could be *given a row, at which column do we have to place its queen*.

- For an $n \times n$ board, how many options do we have for row 0?

From the problem statement we derive that there must be **exactly one queen** at each row, so our approach could be *given a row, at which column do we have to place its queen*.

- For an $n \times n$ board, how many options do we have for row 0?

We have n options, one for each cell in the row 0.

- And for row 1?

From the problem statement we derive that there must be **exactly one queen** at each row, so our approach could be *given a row, at which column do we have to place its queen*.

- For an $n \times n$ board, how many options do we have for row 0?

We have n options, one for each cell in the row 0.

- And for row 1?

We have $(n-1)$ options for each of the n options of row 0, that is $n \cdot (n-1)$.

Can you see where this is going?

Designing an algorithm

The first aspect of the algorithm that we should decide is how to represent the placed queens.

In our case, we will use an array of integers Q. It will represent that position $(i, Q[i])$ contains a queen (i.e. $Q[i]$ indicates which square in row i contains a queen).

Try to think a recursive algorithm that receives the array Q and n as parameter and returns a valid solution for the n-queens problem.

You can use the same idea that we discussed in the previous slide.

Algorithm 1 The `N-queens` recursive algorithm.

Precondition: Q an array of integers representing the position of the queen at each row i. n an integer representing the size of the board.
Postcondition: Q contains a valid solution to the n-queens problem or $NULL$ if no solution exists.

```
procedure NQueens(Q, n)
    if len(Q) = n then return Q
    i ← len(Q)
    for j in 0..(n − 1) do
        if isValid((i, j), Q) then
            solution ← NQueens(Q + [j], n)
            if solution ! = NULL then return solution
    return NULL
```

Complexity of the algorithm?

Algorithm 2 The `N-queens` recursive algorithm.

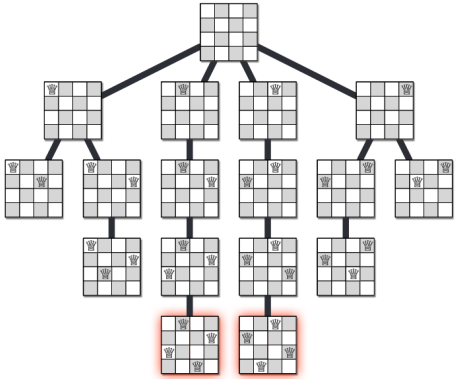
Precondition: Q an array of integers representing the position of the queen at each row i. n an integer representing the size of the board.
Postcondition: Q contains a valid solution to the n-queens problem or $NULL$ if no solution exists.

```
procedure NQueens(Q, n)
    if len(Q) = n then return Q
    i ← len(Q)
    for j in 0..(n − 1) do
        if isValid((i, j), Q) then
            solution ← NQueens(Q + [j], n)
            if solution != NULL then return solution
    return NULL
```

Complexity of the algorithm?

$O(n!)$

This algorithm can be illustrated using a **recursion tree**. In the following example we show the recursion tree for $n = 4$:



Recursion tree for 4-queens

- Each node in this tree corresponds to a recursive subproblem, and thus to a legal partial solution.
- Edges in the recursion tree correspond to recursive calls.
- Leaves correspond to partial solutions that cannot be further extended, either because there is already a queen on every row, or because every position in the next empty row is attacked by an existing queen.

Let's analyze the algorithm more in depth:

Algorithm 3 The `N-queens` recursive algorithm.

Precondition: (...)
Postcondition: (...)

```
procedure NQueens(Q, n)
    if len(Q) = n then return Q
    i ← len(Q)
    for j in 0..(n − 1) do
        if isValid((i, j), Q) then
            solution ← NQueens(Q + [j], n)
            if solution != NULL then return solution
    return NULL
```

- When we exhaust all the possible j for a certain level and found no solution, we return $NULL$.
- This does not mean that the instance has no solution at all! It might happen that there is no solution at some point of our serach, but at other point there is!
- This is why we check if $solution! = NULL$ in the recursive case. We will return $NULL$ **only when there is no possible solution at all** (i.e. all the possible brances at the root return $NULL$).

Backtracking

The recursive pattern that we used to solve the n-queens problem is called **backtracking**. Let's study the general characteristics of this pattern.

- Backtracking algorithms are commonly used to make a sequence of decisions, with the goal of building a recursively defined structure satisfying certain constraints.
- Often (but not always) this goal structure is itself a sequence. For example, in the n-queens problem, the goal is a sequence of queen positions, one in each row, such that no two queens attack each other. For each row, the algorithm **decides** where to place the queen.
- In each recursive call to the backtracking algorithm, we need to make **exactly one decision**, and our choice must be **consistent** with all previous decisions.
- Thus, each recursive call requires not only the portion of the input data we have not yet processed, but also a suitable summary of the decisions we have already made.
- For the sake of efficiency, the summary of past decisions should be as small as possible. In the case of the n-queens problem, we need to keep track of *all the decisions that we took so far*, but this might not be the case for other problems.

Practical case: the 0-1 Knapsack Problem

In the **knapsack problem** we are given a set of items, each with a weight and a value, and we must determine which items to include in the collection so that the total weight is less than or equal to a given limit and the total value is as large as possible.

Illustration for the Knapsack Problem

The **0-1 knapsack problem** restricts the number of copies of each kind of item to **zero** or **one**.

As you know, this problem is an *optimization problem*. We can no longer solve it using a *greedy* approach as we did in the exercises of the subject for the *fractional knapsack problem*.

Therefore, we will design a backtracking algorithm.

Designing an algorithm

- We will start by sorting the items by its *profit to weight* ratio, as we did in the *fractional knapsack problem*.
- In our exploration, we will assume that choosing the **left** child of a node means that we **include** an item.
- Choosing the **right** child of a node means that we **exclude** an item.
- We must keep track of the *best solution* found so far, as we cannot stop searching until we find the *best overall solution*.

Precondition: $items$ a list of items in the form $\langle weight, value \rangle$ and $capacity$ the maximum capacity allowed.

Postcondition: $selected$ a bit mask with the selected items such that the sum of their value ($profit$) is maximal and the sum of their weights is $\leq capacity$, and $profit$ the profit produced by these items.

Algorithm 4 The `0-1 knapsack problem` recursive algorithm.

procedure KNAPSACK($items$, $capacity$)
  Sort $items$ according to their profit to weight ratio.
  return $KnapsackRec(items, capacity, [])$
procedure KNAPSACKREC($items$, $capacity$, $selected$)
  ▷ *Base case*
  if We reached $capacity$ with the items in $selected$ **or** $len(selected) = len(items)$ **then**
    return $selected$, sum of the values of $selected$ items
  $\langle best\_selected, best\_value \rangle \leftarrow \langle [\,], 0 \rangle$
  $next\_item \leftarrow len(selected)$
  ▷ *If we can include the next element to the knapsack, explore this option*
  if We do not overpass $capacity$ by adding $next\_item$ **then**
    $\langle best\_selected, best\_value \rangle \leftarrow KnapsackRec(items, capacity, selected + [true])$
  ▷ *Explore also the option where we exclude the next item*
  $\langle best\_selected', best\_value' \rangle \leftarrow KnapsackRec(items, capacity, selected + [false])$
  ▷ *Return the solution that yields the best value*
  if $best\_value' > best\_value$ **then return** $\langle best\_selected', best\_value' \rangle$
  return $\langle best\_selected, best\_value \rangle$

The idea of the algorithm is similar than in the *n-queens* problem. The main difference is that, from the two possibilities that we have at each node (*include* the next item or *exclude* it), we choose the one that returned the *best* value.

We also *prune* the branches that are *unfeasible* (i.e., the ones where *including* the next element would *overpass* the capacity).

Complexity of the algorithm?

The idea of the algorithm is similar than in the *n-queens* problem. The main difference is that, from the two possibilities that we have at each node (*include* the next item or *exclude* it), we choose the one that returned the *best* value.

We also *prune* the branches that are *unfeasible* (i.e., the ones where *including* the next element would *overpass* the capacity).

Complexity of the algorithm?

$O(2^n)$

Branch and Bound

In the previous section we saw how to *prune* our search tree. This *pruning* allowed us to *skip* unfeasible branches earlier, without having to explore them completely.

In some problems, we can go even further on the detection of these unfeasible branches. That is, we can use some property of the problem that helps us on **quickly** computing an **upper bound** on the cost/benefit of a partial solution.

If this upper bound (i.e. we cannot do better than this) is worse than the current solution, there is no need to continue exploring this branch, and we can *prune* it.

Branch and Bound is a method for solving optimization problems. The idea is to break them down into smaller sub-problems (as we did with backtracking) and using a **bounding function** to eliminate sub-problems that cannot contain an optimal solution.

As we said, computing this **bounding function** must be *fast*. If not, the whole method would not make sense.

Usually, these bounding functions make some *relaxations* on the problem, as we will see in the next example.

Solving the 0-1 Knapsack Problem using Branch and Bound

To solve the 0-1 Knapsack Problem using Branch and Bound we need to do two important modifications to our *backtracking* solution:

1. We must keep track of the **best** solution found so far.
2. We must design and apply a **bounding function** that, given a partial solution, computes an *upper bound*, which represents the *potential* of this partial solution. If the computed upper bound is worse than the best solution found so far, there is no need to continue exploring this branch.

To effectively apply these modifications it is recommended to transform the backtracking algorithm from recursive to iterative.

Bounding function for the 0-1 Knapsack Problem (I)

Consider the following instance: $capacity = 10$,
$items = \{(2, 40), (3.14, 50), (1.98, 100), (5, 95), (3, 30)\}$.

Imagine that in our *partial solution* we exclude the item $(1.98, 100)$, and that I *magically know* that the current best solution has a value of 235.

Do we need to continue exploring? Which bounding function can we use?

Bounding function for the 0-1 Knapsack Problem (I)

Consider the following instance: $capacity = 10$,
$items = \{(2, 40), (3.14, 50), (1.98, 100), (5, 95), (3, 30)\}$.

Imagine that in our *partial solution* we exclude the item $(1.98, 100)$, and that I *magically know* that the current best solution has a value of 235.

Do we need to continue exploring? Which bounding function can we use?

If we **relax** the constraint that says that **I cannot overpass the capacity of the knapsack**, we can decide the add *all the remaining items to the solution*.

If we do so, we will see that **we will not be able to improve the current best solution** (i.e. the sum of their value is 215). Therefore, we can *prune* this branch at this exact moment.

Why is that an **upper bound**?

Bounding function for the 0-1 Knapsack Problem (I)

Consider the following instance: $capacity = 10$,
$items = \{(2, 40), (3.14, 50), (1.98, 100), (5, 95), (3, 30)\}$.

Imagine that in our *partial solution* we exclude the item $(1.98, 100)$, and that I *magically know* that the current best solution has a value of 235.

Do we need to continue exploring? Which bounding function can we use?

If we **relax** the constraint that says that **I cannot overpass the capacity of the knapsack**, we can decide the add *all the remaining items to the solution*.

If we do so, we will see that **we will not be able to improve the current best solution** (i.e. the sum of their value is 215). Therefore, we can *prune* this branch at this exact moment.

Why is that an **upper bound**?

Because the sum of the weights of these items is 13.14, so we cannot fit them all in a valid solution!

Bounding function for the 0-1 Knapsack Problem (II)

We can improve our bounding function futher. The idea is to *relax* the problem *as little as possible*, so the returned upper bound is closer to the real value.

Consider the same instance: $capacity = 10$,
$items = \{(2, 40), (3.14, 50), (1.98, 100), (5, 95), (3, 30)\}$.

In our partial solution we include the items $\{(2, 40), (1.98, 100), (5, 95)\}$ and exclude the item $\{3.14, 50\}$. In this case, the current best solution has a value of 250.

If we apply the same bounding function that we defined before, we have that the *upper bound* on the value would be 265, so we must explore this branch.

But, can we propose another bounding function that approximates better the upper bound?

Let's assume we do not want to *relax* the capacity of the knapsack. In this case, the sum of weights of our partial solution $\{(2, 40), (1.98, 100), (5, 95)\}$ is 8.98.

Notice that we cannot fit the whole $(3, 30)$ item. What if we try to fit a **fraction** of this item?

Let's assume we do not want to *relax* the capacity of the knapsack. In this case, the sum of weights of our partial solution $\{(2, 40), (1.98, 100), (5, 95)\}$ is 8.98.

Notice that we cannot fit the whole $(3, 30)$ item. What if we try to fit a **fraction** of this item?

We will apply the *fractional knapsack* greedy algorithm to the remaining items! The problem relaxation now considers that we can split items, so the value that we compute with this method will also be an **upper bound** (i.e. we will never be able to reach a better value in a valid solution).

The difference is that this bounding function gives a **better approximation** on the upper bound (i.e. it is closer to the real solution). We will always prefer these kinds of bounding functions.

In the example, we can add 1.02 of weight to our solution, which corresponds to 10.2 of value. This gives us an upper bound of 245.2, which is less than 250, so we can *prune* this branch.

General Branch and Bound scheme

Assuming we are minimizing:

1. Initialize the value of the best solution B to infinity.
2. Initialize a queue to hold a partial solution with none of the variables assigned.
3. Loop until de queue is empty:
   1. Take a node N off the queue.
   2. If N represents a valid solution x and $cost(x) < B$, then x is the best solution so far. Record it and set $B \leftarrow f(x)$.
   3. Else, *branch* on N to produce new nodes N_i. For each of them:
      1. If $bound(N_i) > B$ do nothing, it will never lead to an optimal solution.
      2. Else, store N_i on the queue.

We can use different `queue` data structures. Depending on their implementation (FIFO, LIFO, priority queue…) we will obtain different search strategies (breadth-first search, depth-first search, best-first search…).

We will study more in depth these algorithms in the Artificial Intelligence subject next year!

Bibliography

- Jeff Erickson. *Algorithms*. `http://jeffe.cs.illinois.edu/teaching/algorithms/`. Chapter 2.